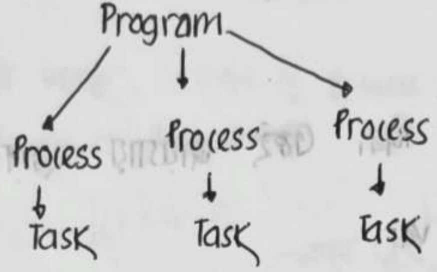
Operating System:

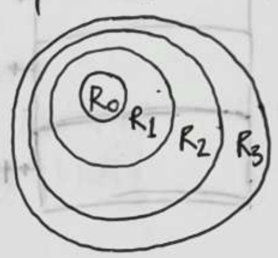
Lec Sameha

Accessibility:


Program
Process
Process
Process
Task
Task
Task

প্রোগ্রামের under এ থাকা Process Task handle করে।

যদি আলাদা ২টি Process কে একই সময় Same Task দেয়; same Power দেওয়া হয় তাহলে তাদের Data Overwrite হবে। → Concept of Accessibility Ring বলা হয়ে থাকে।



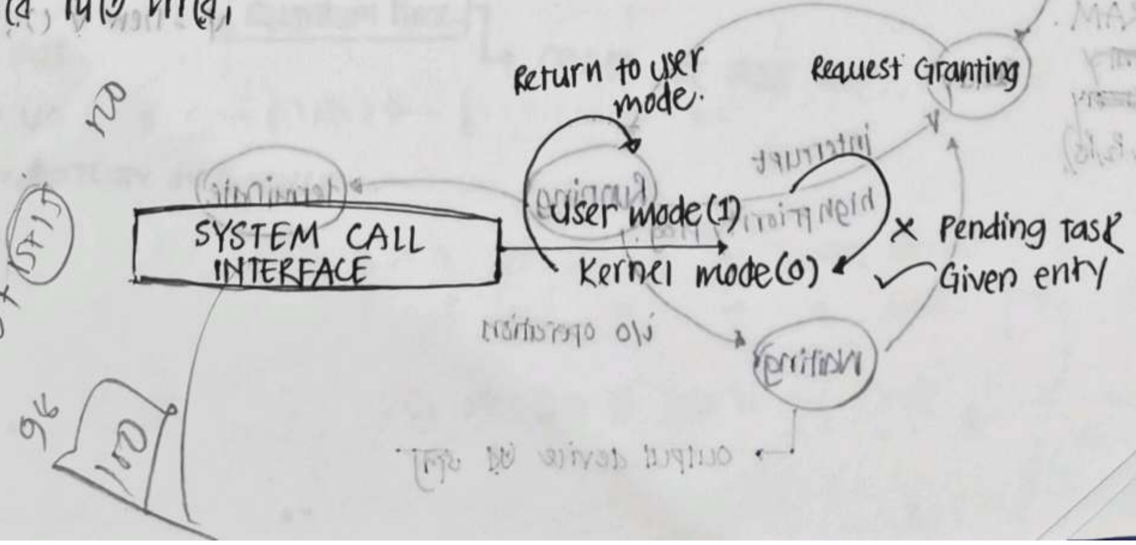
- R_0 → Kernel mode (0 Bit)
  - Bootup system, errorhandling & complex problems.
  - super power.

R_1/R_2 → Medium lvl permission.
- যখন ইচ্ছা Permission দিতে পারব। (System drive). → Windows/Linux এ Available না।

R_3 → Low power (Less accessible state) - (1 Bit)
- User mode. (Check করে Permission দিবে).

can be handled by R_3.

* কেন আমি R_3 রাখতেছি (একটি থাকলে No permission).
- Security concern. R_3 contain 3rd party apps যার মধ্যে Junk আছে। যদি Permission না থাকতো তাহলে OS এর ভেতরে গিয়ে আমার imp file Delet করে দিতে পারে। { HW.
- OS ও Delet করে দিতে পারে।


Return to user mode.
Request Granting
SYSTEM CALL INTERFACE
User mode (1)
Kernel mode (0)
× Pending task
✓ Given enty
120
90+
96

Process: → executable file.

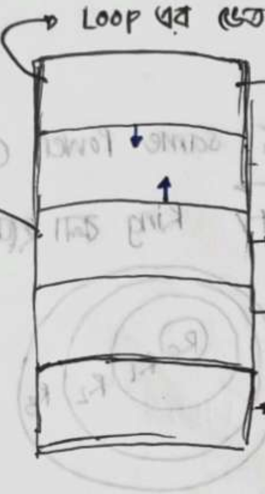
Program এখন run হয়ে execution হয় → Process → Dynamic
→ static

Secondary memory $\xrightarrow{\text{execution}}$ RAM

Process কেন Dynamic?

Program এ এখন User input দিব। একই জায়গায় ২বার ২টা value input দিতে পারি, change হচ্ছে।

→ Loop এর ভেতরের var.



→ stack (Localvar)

free space

→ heap (dynamic memory).

→ data (Global var) → (upper of main func).

→ text (Program code).

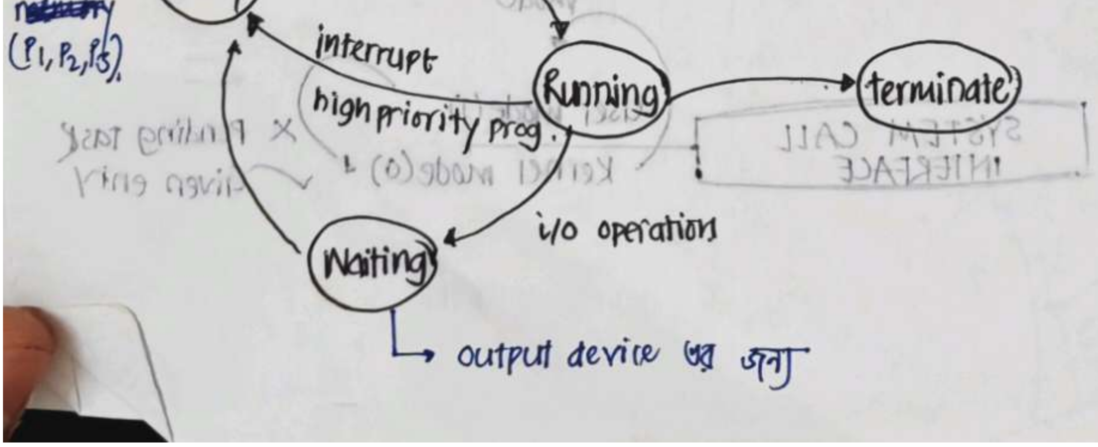
Particular obj create হলে memory alloc হয় (Linked list, Array list, pointer).

কেন? ১টা Program run করলে শুধু ১টা Process On হবে না।

যদি Process crash করে বা Fault না হয় (সফল)

```
int a =5;  → Global var
int b =6;
main ()
for (int i=0; i<n, i++)
{  int p;
   s.o.s = (p);
}
V = new V ();
```

Process State Diagram:


secondary mem
New Process
RAM.
~~Primary memory~~
(P1, P2, P3).
Ready
Scheduler
Running
terminate
interrupt
high priority prog.
i/o operation
Waiting
→ output device এর জন্য

- scheduler determine করবে কখন কোন Process কাজ করবে (OS এ দিবে)
- যদি Process এর 3rd party resource লাগে → Waiting (phase এ থাকবে) → Ready
- Running → Terminate.

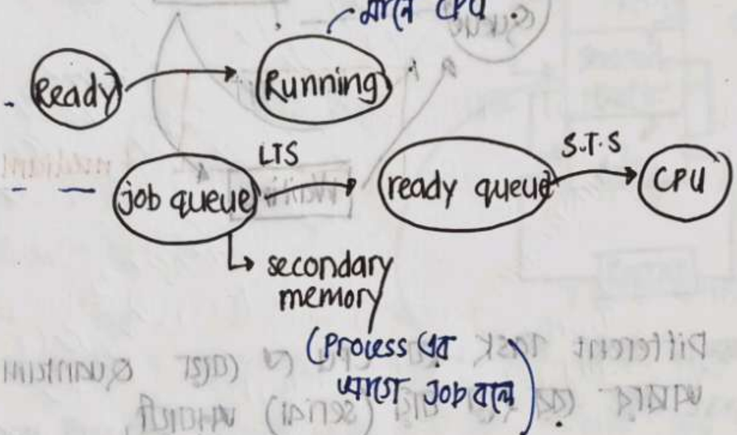
∴ P_3 → Running state এ আছে, তখন P_4 Process (high Priority নিয়ে আসলে) P_3 কে interrupt করায় Ready তে নিয়ে রাখে।

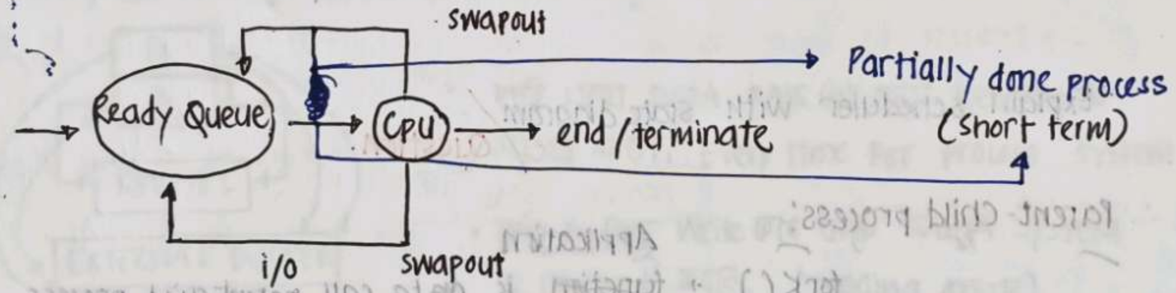
Scheduler:

① Short term ----

Ⓘ Long term ----

Ⓘ Medium term


মানে CPU
Ready
Running
LTS
S.T.S
job queue
ready queue
CPU
secondary memory
(Process এর লাগে Job বলে)


swapout
Partially done process
(Short term)
Ready Queue
CPU
end / terminate
i/o
swapout

context switch/ Quantum time.

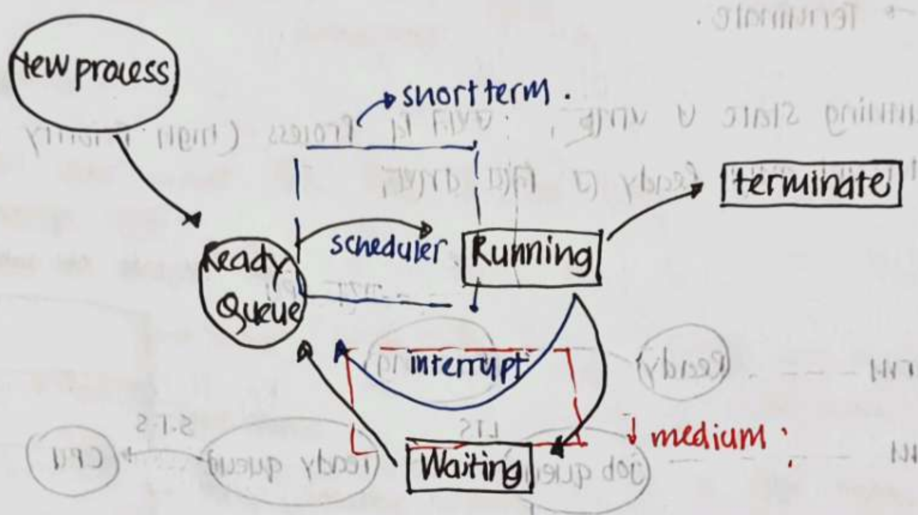
P_1 কে ২ মিনিট দিলাম কিন্তু ওর ২ মিনিট ১০ সেকেন্ড লাগে। এখন যদি ওকে বের করে নিয়ে আসি swap out.

Quantum time. → CPU তে যেই সময় লাগে।

- i/o
- Partially done process

pe → প্রত্যেকের একটি unique process id থাকে।

Process State Diagram:

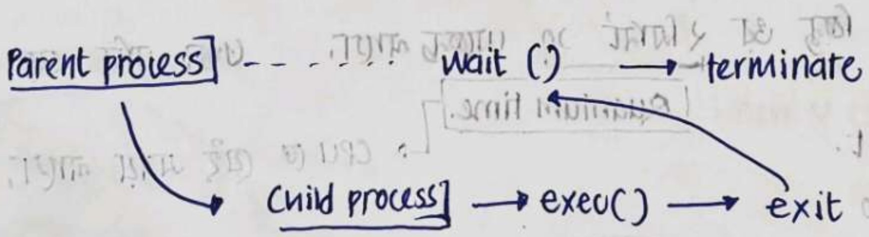

New process
short term.
terminate
Ready Queue
scheduler
Running
interrupt
Waiting
medium ;

Context switch: Different Task কে CPU কে দেয়ে Quantum Time থেকে আবার বের হয়ে যায় (serial) অনুযায়ী।

. Explain scheduler with state diagram / Question.

. Parent-Child process:

Application
fork() → ~~function~~ is an to call parent-child process .
exec() → child এর execution কে বুঝায়
wait () → Parent process wait করবে


Parent process
wait ()
terminate
Child process
exec()
exit

Parent process এর Process id 0 থেকে বড়
Child " " " " 0.

fork() → id দেয় 0 or 0 থেকে বড়।

Zombie process: যদি Child process execute করতে পারে না, Parent process ও wait থেকে বের হতে পারবে না, Terminate হবে না।

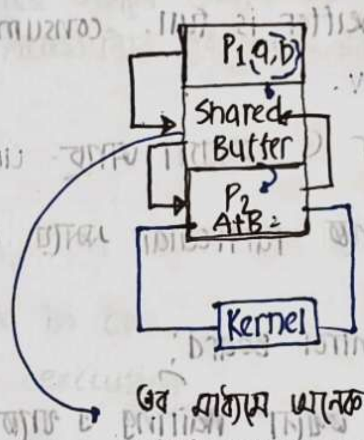
- Interruption কল করে বের করে আনব
  ↳ Parent process কে।

(IPC) · INTER PROCESSING.
Communication:

i) Independent process

ii) Co-operative process.

Share Memory.

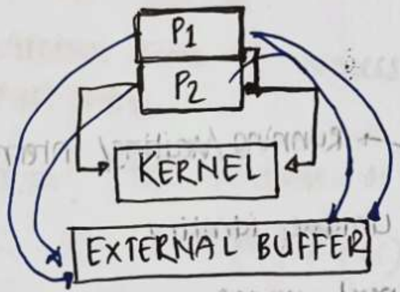


যদি shared Access থাকে, এমনভাবে দিব থাকে Both can not read & write at the same time.

এর মাধ্যমে অনেক বড় DATA Exchange হয়
- শুরু হবার ১বারে SYSTEM CALL লাগে।

MESSAGE PASSING:

Message Passing

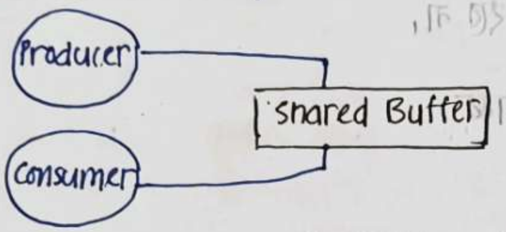


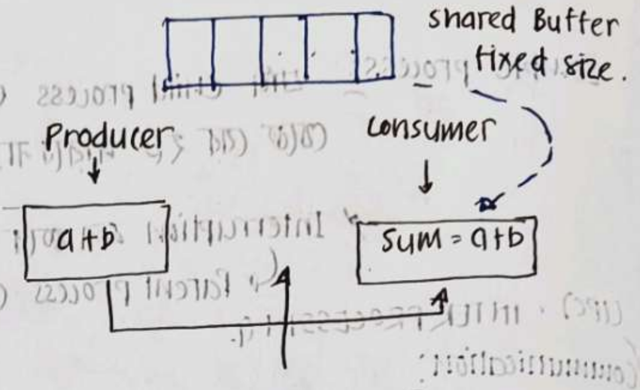
- প্রতি সময় DATA PASS এর জন্য KERNEL এ যাওয়া লাগে। Every time for process system call
- যদি P_1 কিছু write করে থাকে তাহলে SYSTEM P2 কে hold করবে. (logging বাড়বে)
- No big data utilize.
- Do certain task through msg pass

* exam difference between this two.
pros & cons

i. Information sharing
ii. computation spd.
iii. Modularity.

production / consumer - এক হচ্ছে না।


Producer
Shared Buffer
Consumer


shared Buffer
fixed size.
Producer
consumer
a+b
sum = a+b

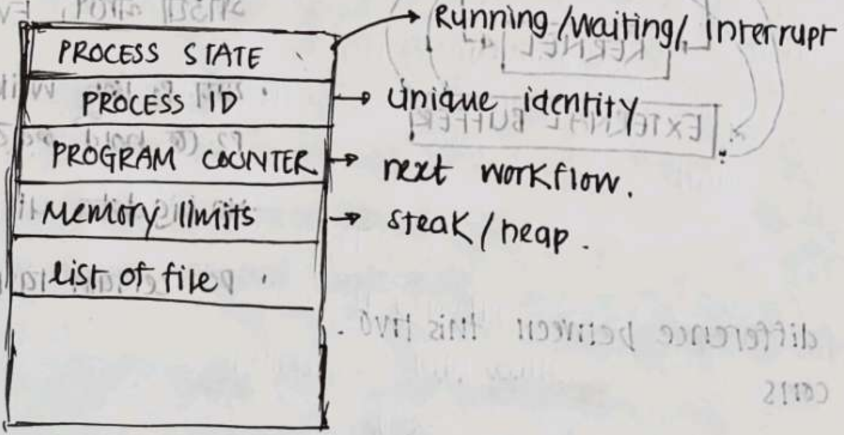
* if shared ~~b~~uffer is full. consumer সরবে না। সে বারবার search করবে overflow.

Shared Buffer এ জায়গা থাকে underflow.

যদি A, B থাকে particular সময়ে shared Buffer এ কাজ শেষ করছে - Override.

PCB: Process Control Board:

- P1 প্রসেস যদি এখনো waiting এ থাকে • তাহলে সেটার ~~Memory এ~~ executed portion এর info থাকতে হবে। সেটা write করে রাখা যায়।
  - Process Id save থাকে
  - Process State


PROCESS STATE
PROCESS ID
PROGRAM COUNTER
Memory limits
List of file
Running / waiting / interrupt
unique identity
next workflow.
steak / heap.

P1.

Code	Data	File
reg	reg	reg
Prog ~~cell~~ counter	"	"
Stack	"	"

∴ Lagging হবে না।

যদি spd বেশি করতে চাই

Particular ~~process~~ task কে thread এ ভাগ করব

- Thread গুলো Process এর info use করবে। NO memory Addressing.

- Parent-Child আনলে computational power কম হবে।
- আলাদা Access দিতে হবে (Info ও দিতে হবে)।

Arrival Time: When process Arrives in system.

Burst Time: How much time process Spends in CPU.

Completion ": When a process is done with execution

TurnAround ": The time from arrival to completion

Waiting ": WT = TAT - BT.

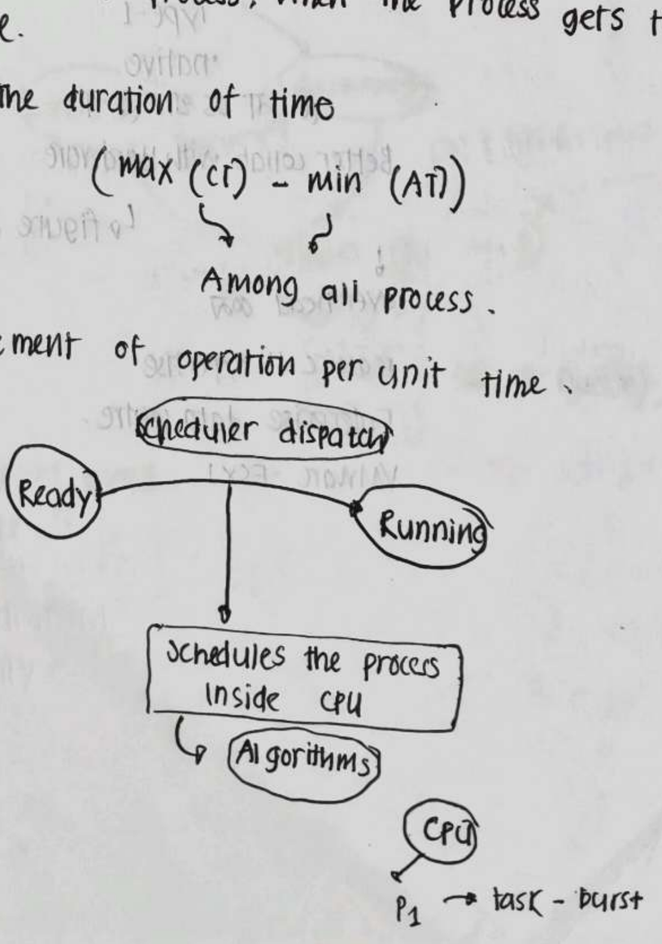
Response ": Arrival Time of process, When the process gets the CPU for very first time.

Scheduling length: The duration of time

$$(\max(CT) - \min(AT))$$

Among all process.

Througput: Measurement of operation per unit time.


Scheduler dispatch
Ready
Running
Schedules the process inside CPU
Algorithms
CPU
P1 → task - burst

threading.

Preemptive → waiting stage.

→ CPU এর ভেতর কাজ শেষ করতে যত সময়

Process	Arrival time	Burst Time	Completion Time.	Turn Around (CT-AT)	Waiting Time (TAT-BT)
P_1	0	30	30	30	0
P_2	0	5	35	35	30
P_3	0	5	40	40	35

Avg TAT = 35

Avg WT = 21.66.

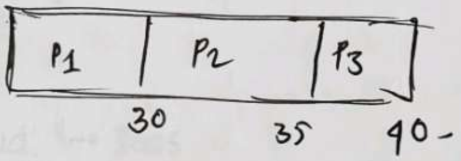
Scheduling Length = Max (CT) - Min (AT)

= 40 - 0

= 40.

FCFS → Firstcome first serve.

Throughput = $\frac{\text{No of process}}{\text{S.L}}$ = 3/40.

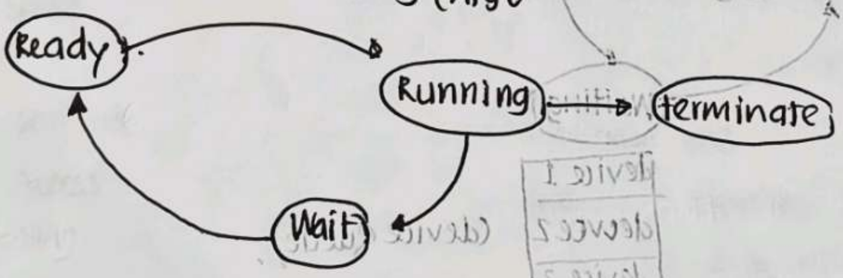


Gantt chart

Non preemptive

- Depend on process arr
- If same AT go for process ID.

Scheduling (Algo)



Preemptive - interruption, context switch.

AT	BT	CT	TAT	WT
0	4	4	4	0
1	2	6	5	3
2	3	9	7	4
3	5	14	11	6
4	6	20	16	10
5	1	21	16	15

36
+7
43
+16
59

59/6

38/6

6|59|9.83
54
50
48
20

6|38|6.3
36
20

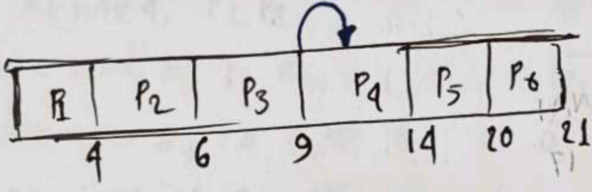
6|21|3.5
18
30

9.83

6.33

scheduling = 21 - 0
= 21

Throughput = 6/21
= 3.5

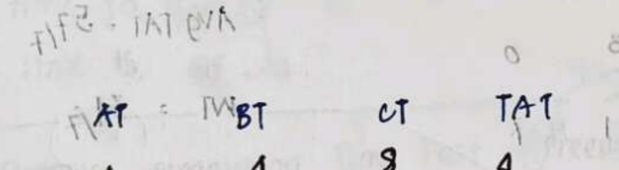

P1
P2
P3
P4
P5
P6
4
6
9
14
20
21

	AT	BT	CT	TAT	WT
P_1	4	4	8	4	0
P_2	8	2	23	15	13
P_3	6	3	14	8	5
P_4	5	3	11	6	3
P_5	2	1	3	1	0
P_6	7	7	21	14	7

∴ AVG TAT : 48/6 = 8

Avg WT : 28/6 = 4.6

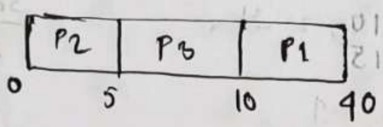
Scheduling length : 21


P5
P1
P4
P3
P6
P2
0
2
3
4
8
11
14
20
23

Burst time ছোটো তারা আসছে কিন্তু আগেরটার জন্য loop এ পড়ে আছে = **convoy effect**
↳ FCFS.

∴ Problem solve এর জন্য smaller burst time priority পাবে,

P_1	0	30	40	40	10
P_2	0	5	5	5	0
P_3	0	5	10	10	5


P2
P3
P1
0
5
10
40

SJF

40?

$\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$

CT - BT

$\frac{x_1}{x_2} = -\frac{y_1}{y_2}$

TAT - AT

CT-AT

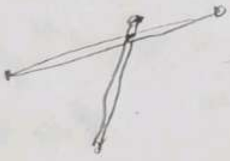
TAT $x_1 y_2 - B + x_2 y_1$

WT = TAT - B

Queue!

at time 0 P_1, P_2, P_3

at time 5 P_1, P_3



Non preemptive.

	AT	BT	CT	TAT	WAT
P_1	0	6	23	23	17
P_2	0	3	3	3	0
P_3	1	4	12	11	7
P_4	2	2	6	4	2
P_5	3	1	4	1	0
P_6	4	5	17	13	8
P_7	6	2	8	2	0

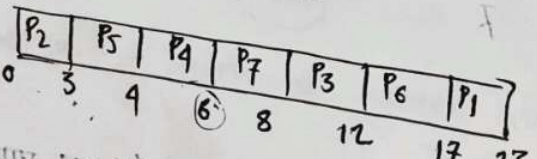
Avg TAT = 57/7

WT = 34/7

SL = 23 - 0
= 23

T.O = 7/23

Gantt chart


P2
P5
P4
P7
P3
P6
P1
0
3
4
6
8
12
17
23

Ready Queue.

At time 0. P_1, P_2

At time 3, P_1, P_3, P_4, P_5

At time 4. P_1, P_3, P_4, P_5

At time

SJF - Shortest Job first

	AT	BT	CT	TAT CT-AT	WAT TAT-BT
P1	0	4	4	4	0
P2	4	3	7	3	0
P3	6	1	8	2	1
P4	3	5	15	12	7
P5	8	2	10	2	0

Avg TAT = $23/5 = 4.6$

Avg WAT = $8/5 = 1.6$

SL = 15 - 0
= 15

Througput $\boxed{5/15} = 1/3$

Ready Queue.

At time 0, P1

At time 4, P2, P3

At time 7, P3, P4

At time 8, P4, P5

At time 10, P[illegible] P4

At time 15, [illegible] —

P1	P2	P3	P5	P4

0 4 7 8 10 15

SRTF Shortest remaining Time Fast. (Preemptive) ← Non-preemptive

	AT	BT	CT	TAT	WAT
P1	0	6	17	17	11
P2	1	4	9	8	4
P3	2	1	3	1	0
P4	3	2	5	2	0
P5	4	1	6	2	1
P6	5	3	12	7	4

Avg TAT = $37/6$

WT = $20/6$

SL =

Througput =

At time 0, P1

At time 1, P1, P2

At time 2, P1, P2, P3

At time 3, P4, P1, P2

At time 4, P4, P1, P2, P5

At time 5, P4, P1, P2, P5

At time 6, P5, P1, P2

P1 → ~~6~~ 5

P2 → ~~4~~ 3

P4 → ~~2~~ 1 → ~~P5~~

~~P5~~ →

P1	P2	P3	P4	P5	P2	P6	P1

0 1 2 3 5 6 9 12 17.

SJF → non preemptive

SRTF → preemptive

OS

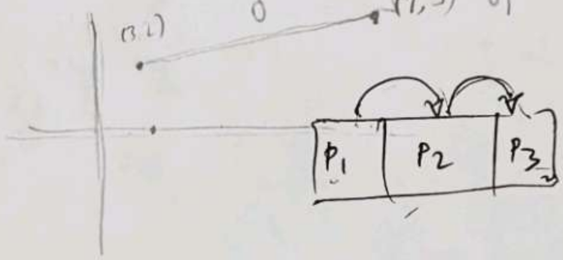
G

∴ SRTF এ starvation problem.

↳ Priority দিয়ে দমাতে না।

RT : Arrival time - The time when it gets CPU

Context switching : 7.



P1	0	10
P2	2	20.
P3	6	30.

(x1, -x2)

P

0, 3
2, 6
4 4
6, 5
8, 2

0, P1 ~~10~~ 9
1, P1 ~~9~~ 8
2, P1 - ~~8~~ 7
3 P1 - ~~7~~ 6

non → premptive.

	AT	BT	CT	TAT	WT	RT
P1	0	3	3	3	0	
P2	2	6	9	7	1	
P3	4	4	13	9	5	
P4	6	5	20	14	9	
P5	8	2	15	7	5	

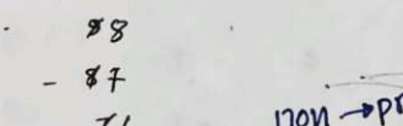
Avg TAT = 40/5

Avg WT = 20/5

$HRR = \frac{w+s}{s}$

$\frac{w}{s} + 1$

$\frac{2CJ-A}{S}$



HRRN: highest response ration next.

At time 9s.

RR [P3] · $\frac{[9-4]+4}{4} = 2.25$

RR [P4] $\frac{[9-6]+5}{5} = 1.3$.

RR [P5] : $\frac{(9-8)+2}{2} = 1.5$

RR ∝ S^{-1}

∴ S বাড়লে RR কমে।

RR ∝ W.

RR ∝ W

RR ∝ S^{-1}

	AT	BT	CT	TAT	WT	RT
P1	0	12	27	~~16~~ 27	15	0
P2	2	4	6	4	0	0
P3	3	6	12	9	3	3
P4	8	5	17	9	4	4

Avg WT = 22/4

TAT = 49/4

0 → P1 ~~12~~ 11
1 - P1 ~~11~~ 10
2 → P2 ~~4~~ ~~5~~ 3
3 → P3 ~~6~~ 5
4 → P3 ~~5~~ 4
5 → P3 ~~4~~ 3
6 → P3 → ~~3~~ 2
7 - P3 - ~~2~~ 1
8 →

0 → ~~12~~ P1
2 → 4P2, 10P1
3 → 6P3, 3P2, 10P1
~~8~~ 6 → 6P3, 10P1
~~12~~ 8 → 10P1, 4P3, 5P4
12 → 10P1, 5P4
17 - 10P1.
27.

P1	P2	P2	P3	P3	P4	P1

0 2 3 6 8 12 17 27

Priority based → Preemptive
→ Non-preemptive. (static)

Dynamic → process 2min wait · priority (+2).

Process	AT	BT	Priority	CT	TAT	WT
P1	0	4	4	4	4	0
P2	1	2	5	18	17	15
P3	2	3	6	16	14	11
P4	3	1	10	5	2	1
P5	4	2	9	7	3	1
P6	5	6	7	13	8	2

FT

4/6 3/6

TAT=8

AVG TAT= 8
AVG WT = 5

13 JUL

Ready Queue: 0 → P1
4 → P2, P3, P4, P5
5 → P2 P3 P5 P6

P1	P4	P5	P6	P3	P2

0 4 5 7 13 18 18

AWT = 25/4

TAT = ~~38~~48/4 = 12,

	AT	BT	Priority	CT	TAT	WT
P1	0	6	2	6	6	0
P2	4	10	1	16	~~6~~12	2
P3	4	4	2	~~19~~23	19	15
P4	8	3	1	19	~~14~~12	8
P5						
P6						

Ready

P1	P2	P4	P3

0 6 16 19 23

Preemptive: (Priority Queue):

Process	Arrival Time	Burst time	Priority	CT	TAT	WAT
P1	0	4	4	18	18	14
P2	1	2	5	15	14	12
P3	2	3	6	14	12	9
P4	3	1	10(H)	4	1	0
P5	4	2	9	6	2	0
P6	5	6	7	12	7	1

Priority Queue:

P1	P2	P3	P4	P5	P6	P3	P2	P1	
0	1	2	3	4	6	12	14	15	18

Ready Queue:

0 → 4P1
1 → 3P1, 2P2
2 → 3P1, 1P2, 3P3
3 → 3P1, 1P2, 2P3, 1P4 → X
4 → 3P1, 1P2, 2P3, 2P5
5 → 3P1, 1P2, 2P3, 1P5, 6P6
6 → 3P1, 1P2, 2P3, 6P6
12 → 3P1, 1P2, 2P3.
14 → 3P1, 1P2.
15 → 3P1
18 → $\boxed{X}$

Process	AT	BT	Priority	CT	TAT	WAT (TAT-BT)
P1	0	7	6	27	27	20
P2	1	4	5	18	16	12
P3	2	6	3	15	13	7
P4	3	2	1(H)	5	2	0
P5	4	3	5	21	17	14
P6	6	5	2	11	5	0

P1	P2	P3	P4	P5	P6	P3	P2	P5	P1

0 1 2 3 5 6 11 15 18 21 27

28
40
40
88

109
9

Ready Queue:

0 → 7P1
1 → 6P1, 4P2
2 → 6P1, 3P2, 6P3
3 → 6P1, 3P2, 5P3, 2P4
4 → 6P1, 3P2 5P3, 1P4, 3P5
— 5 → 6P1, 3P2, 5P3, 3P5 (P4×)
6 → 6P1, 3P2, 4P3, 3P5, 5P6
→ 11 → 6P1, 3P2, 4P3, 3P5
→ 15 → 6P1, 3P2, 3P5
→ 18 → 6P1, 3P5
21 → 6P1
27 → ,

Aging (কোনো Process
3 momentum পর্যন্ত Wa
করালে Priority 1 করে U

না বললে static হবে.

Round-Robin: (Practicality এর দিক দিয়ে → এখন 1
(Timesharing এর perspective full fill করে)
· Quantum time এর পর context switch.

Process	AT	BT	Priority	Completion time	TAT	WT
P_1	0	3	1	9	6	6
P_2	0	6	4	17	11	2
P_3	0	4	2	13	9	4
P_4	0	5	3	18	13	6
P_5						

105
5

$\boxed{Q=2}$

P_1	P_2	P_3	P_4	P_1	P_2	P_3	P_4	P_2	P_4

0 2 4 6 8 9 11 13 15 17 18

0 → $3P_1, 6P_2, 4P_3, 5P_4$
2 → $1P_1, 4P_2, 4P_3, 5P_4$
4 → $1P_1, 4P_2, 4P_3, 5P_4$
6 → $1P_1, 4P_2, 2P_3, 5P_4$
8 → $1P_1, 4P_2, 2P_3, 3P_4$
9 → $4P_2, 2P_3, 3P_4$
11 → $2P_2, 2P_3\ 3P_4$
13 - $2P_2, 3P_4$
15 - $2P_2, 1P_4$
17 - $1P_4$.

	AT	BT
P_1	0	4
P_2	1	5
P_3	2	6
P_4	3	3
P_5	4	2
P_6	5	4.

$$\begin{array}{r} 63 \\ +15 \\ \hline 78 \\ +11 \\ \hline 89 \end{array}$$

$$\begin{array}{r} 40 \\ +26 \\ \hline 66 \\ 28 \\ \hline 94 \\ 7 \\ \hline 101 \\ +11 \\ \hline 112 \\ +17 \\ \hline 129 \end{array}$$

$$\begin{array}{r} 20 \\ +15 \\ \hline 35 \\ +21 \\ \hline 56 \\ 4 \\ \hline 60 \\ +22 \\ \hline 82 \\ +4 \\ \hline 86 \end{array}$$

80

$$\begin{array}{r} 63 \\ +15 \\ \hline 78 \\ +23 \end{array}$$

0 → $4P_1$
2 → $2P_1, 5P_2, 6P_3$
4 → $2P_1, 3P_2, 6P_3, 3P_4$ $2P_5$
6 → $2P_1, 3P_2, 4P_3, 3P_4, 2P_5, 4P_6$
8 → ~~2P1~~, $3P_2, 4P_3, 3P_4, 2P_5, 4P_6$
10 → $1P_2, 4P_3, 3P_4, 2P_5, 4P_6$
12 →

0 → $4P_1$
2 → $5P_2, 6P_3, 2P_1$
4 → $6P_3, 2P_1, 3P_4, 2P_5, 3P_2$
6 → $2P_1, 3P_4, 2P_5, 3P_2, 4P_6, 4P_3$
8 → $3P_4, 2P_5, 3P_2, 4P_6, 4P_3$
10 → $2P_5, 3P_2, 4P_6, 4P_3, 1P_4$
12 → $3P_2, 4P_6, 4P_3, 1P_4$
14 → $4P_6, 4P_3, 1P_4, 1P_2$
16 → $4P_3, 1P_4, 1P_2, 2P_6$
18 – $1P_4, 1P_2, 2P_6, 2P_3$
⑲ → $1P_2, 2P_6, 2P_3$
20 → $2P_6, 2P_3$
22 → $2P_3$
24 → ✗

very very small
small
QT Large (less interaction)
very very large (FCFS)
↳ প্রসেসের পর পর switching
(AT)

Q = 3.

	AT	BT	CT
P_1	5	6	~~28~~ 26
P_2	8	7	32
P_3	5	8	28
P_4	6	3	16
P_5	2	2	4
P_6	4	9	23

P_5	P_3	P_6	P_1	P_4	P_3	P_2	P_6	P_1	P_3	P_2

2 4 7 10 13 16 19 22 25 26 28 32

0 →.

2 → $2P_5$ →

4 → $8P_3$, $4P_6$

7 → $4P_6$, $6P_1$, $3P_4$, $5P_3$

10 → $6P_1$, $3P_4$, $5P_3$, $7P_2$, $1P_6$

→ 13 → $3P_4$, $5P_3$, $7P_2$, $1P_6$, $3P_1$

16 → $5P_3$, $7P_2$, $1P_6$, $3P_1$

19 → $7P_2$, $1P_6$, $3P_1$, $2P_3$

22 → $1P_6$, $3P_1$, $2P_3$, $4P_2$

→ 23 → $3P_1$, $2P_3$, $4P_2$

→ 26 → $2P_3$, $4P_2$

28 → $4P_2$

32 → λ

Process Synchronization:

$P_1 \rightarrow a{+}{+}$ (1) → 2   যদি a = 2 print করে

$P_2 \rightarrow a{-}{-}$ (2) → 1. → a = 1,

share variable updated হলে তবে।

$\boxed{P_1, P_2}$ → shared Buffer

overflow → producer] → race-condition (issue).
underflow → consume

Process synch.

- → Entry sec (ES)
- → Critical sec (CS)
- → Exit sec (ES)
- → Remainder sec (RS

```
int a
| int a = 1        int a = 2 ]  → CS
| a++              a--       ]
```

```
| if β = 2   |  → Remainder sec.
| print      |
```

(P1, P2)

(P3, P4)

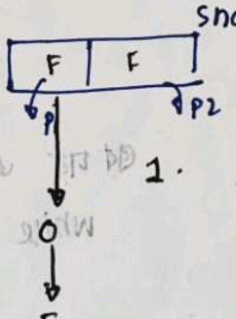
- Mutual exclusion → Particularly ১টা Process কেই Access দিবে।
- Progress → তারা CS এ যাবে তাদের নিজে করবে
- Bounded waiting → যদি কাজ শেষ করে দেবার পর যেতে হয় তাহলে Timeline set করে দিব। P3 শেষে আরও ১টা Process in হবে। এর পর আবার P3 যাবে।

Mutual exclusion: Peterson's Algorithm;

0

(P1)

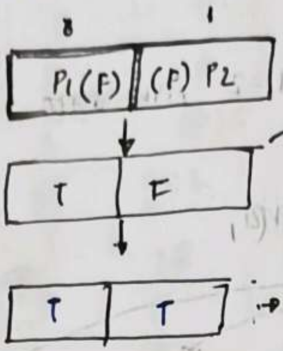
```
do{ flag[0] = true;
    turn = 1          F              T
    while (flag[1] && turn == 1);
    //CS                 (F && T)
    flag[0] = false.   ← CS
    //RM
}
while(turn)
```



(P2)

flag[1] = true.
turn = 0.

তাই একসাথে Access করে।



→ 0 number Index কে True করে।

এখন P2 চাইবে in হবে →

→ যখন দুইটা True হবে while loop এ আটকায় যাবে কারণ আমরা Always Opposite Process নিয়ে কাজ করে।

drawback:

- বারবার while loop এ আটকায় যাচ্ছে
- Only ২টা Process নিয়ে কাজ করে
- ২টা জিনিস Flag & Turn.

Test & set:

```
boolean test-and-set (boolean *target)
 boolean rv = *target
  *target = true;
   return rv;
}
```

Lock = false. → target এর value ও false → True করবে।

P1

```
do{
while (test_and-set (&lock))
//do nothing
// CS
lock = false.
// RM
} while (True)
```

rv
False

lock | F | T
Target | T

rv | false.

এর পর অন্য Process আসলে Target True হয়ে যাবে while এ ঘুরবে।

compare & swap:

By default
&lock = 0
0 0 1

```
int compare_and_swap (int *value, int expected, int newvalue)
        int temp = * value;
      if (* value = expected)
          * value = new_value
          retun loop;
      }

   do {
      while (compare - and- swap ( &lock, 0, 1) != 0)
          /* CS */
       lock = 0
       /* RM */
     }
     while (true).
```

] false হলে CS এ.
True হলে Loop এ.

M~~dex~~ut Lock:

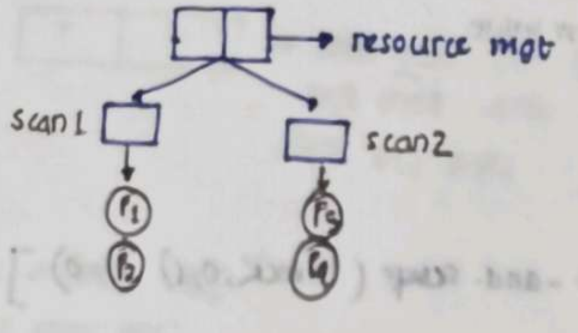
initial available = true;

```
        acquire ()
           release ();

  acquir ()
  {
    while (! available)
      { /* do nothing */
       available = ~~true~~; false;

    }
  release ()
    {
     available = true;
  do {
       acquire ()
         CS
        release ()
           RM
  }
  while (true).
```

বার বার - Deadlock.

Busy-Waiting: এক Resource কে পাওয়ার জন্য বার বার request করা,

ধরি CS এর মধ্যে ২টা /৩টা /৪টা Process আছে.


resource mgt
scan1
scan2
P1
P2
P3
P4

Semaphore: আগে থেকে declare করব আমার কাছে কয়টা Resource আছে।

```
wait () {
  while (s<=0)
      //* busy waiting//
} s--

signal(s) {
  s++,
do
{
  wait (s)
  //CS//
  signal (s)
  //RM//
}
  while (true);
```

২টা Process P1, P2 আছে চাই P2 আগে চলুক .

```
Simaphore, S =0
Process 1
  { wait(s) ;
  }
P2
  {
    Signal (s) ;
  }
```

PCB → PID
Space
File .

(POSIX)

(Pthread C func.

Reciepe book → program
Manager → Kernel
Chief → thread
↳ Diary → Thread control Board .
- Thread ID
- PC (next instruction address)
- stack , register thread.

Waiter → User interface / System call

~~Lg2g~~

Lg2aOR

Online
↳ pthread
scheduling Algo .

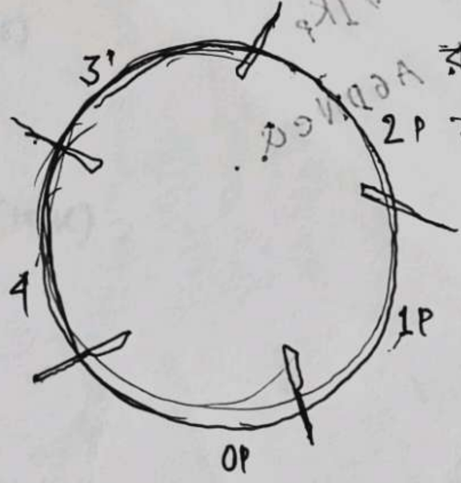
raVJKr

AGDNeQ .

```
Type struct {
    int value ;
  Struct group*list ;
  }
  semaphore

wait ("semaphor*s)
  {  s → value -- ;
  if (s → value <0)
  {
     Add process to
        S → list
     block type ;
   }
  Signal ("semaphor"i *s)
     S → value++
  if  (S → value ≤0)
    { remove S from
      list ;
  }
     wake up
  }
```

```
do {
   wait () ;
   // CS
   Signal ()
   //RM
} while (true) ;
```

Dining philosophy



রুচি কতোটা খারাপ হলে আরেকজনে
2P দেয়া

→ [illegible]
→ [illegible]
→ [illegible]

wait ([illegible])
wait ([illegible])
[illegible]
signal ([illegible])
signal ([illegible])

→ [illegible]

[illegible]
[illegible]

[illegible]
[illegible]

→ no of sectors in disk = no of [illegible] × no track × no sectors per surface

Address bits = $\log_2$ (no of sectors in disk) [illegible]

Disk capacity = no of platters × no of tracks per surface × no of sectors per track × sector capacity

Disk access time = seek time + rotational latency (rotation of time) + sector transfer time + additional delay

Avg rotation of disk surface → 1/2 disk rotation

1 sector transfer time = 1 disk rotation time / no of sectors per track

File → name, type, extension, author name, size, date
↳ system → FAT32 → [illegible] NTFS.

swap → swap space (main memory [illegible]).

* File directory → can not have same file name for more than one user.

DOS → root directory to [illegible] (no access)